

Booqly

Cross-Platform Reading Companion App

Presentation Guide for Your Professor

Everything you need to explain the project: purpose, features, architecture, technologies, demo flow, and likely questions.

Generated: June 09, 2026

Repository: github.com/AliRaya26/BooqlyProject

Firebase Project: booqlyapp-83777

1. Elevator Pitch (30 seconds)

Booqlly is a cross-platform mobile and web application built with Flutter that helps users discover books, manage a personal library, track reading progress, read PDFs in-app, get AI-powered recommendations, and receive smart reading reminders based on their Google Calendar free time.

The app solves a real problem: readers lose track of what they are reading, struggle to find their next book, and lack motivation to read consistently. Booqlly combines a modern UI, cloud backend (Firebase), and AI (Google Gemini) into one integrated reading companion.

One-sentence summary

"Booqlly is a Flutter-based reading companion that uses Firebase for auth and data, Gemini AI for personalized book chat, and Google Calendar to schedule reading reminders during free time."

2. Problem Statement & Solution

Problems addressed

- Scattered reading lists across notes, apps, and memory
- No single place to track progress across multiple books
- Difficulty discovering books aligned with personal interests
- Low motivation and no habit-building support
- No intelligent assistant to answer questions about owned books

Our solution

- Unified library with Reading / Want to Read / Completed tabs
- In-app PDF reader with page-level progress sync to the cloud
- Genre-based onboarding and Discover feed for recommendations
- AI chat grounded in the user's library, preferences, and reviews
- Calendar-aware local notifications during free time slots

3. Main Features to Demonstrate

Authentication & Onboarding

- Email/password sign-up with 6-digit email verification
- Google Sign-In (one-tap login)
- Forgot password via Cloud Function + Gmail SMTP
- First-time onboarding: pick genres, reading theme, and pace

Home Dashboard

- Continue Reading card with live progress
- Want to Read carousel
- Reading streak calendar and monthly stats
- Bottom navigation: Home, Discover, Add, Friends, Settings

Library & Discovery

- Personal library with category filters and real-time Firestore sync
- Search catalog by title or author
- Book detail: progress slider, favorites, reviews, mark completed
- Full-screen PDF reader (Syncfusion PDF viewer)

AI Assistant (Ask AI)

- Powered by Google Gemini API
- Recommends books based on genres, library, and ratings
- Can identify books from uploaded cover photos
- Answers questions like ratings, summaries, and what to read next

Social & Goals

- Friends feed: follow users, see reading activity
- Reading goals: yearly and daily targets with progress tracking
- Reading Wrapped: year-in-review stats and highlights
- Book notes and friend notes sharing

Settings & Integrations

- Google Calendar linking for free-time detection
- Reading reminder notifications during calendar gaps (25+ min)
- Light / Dark / System theme with saved preference
- Profile management and sign out

4. Technology Stack

Layer	Technology
Frontend	Flutter (Dart 3.11+), cross-platform: Android, iOS, Web, Windows
Backend	Firebase Auth, Cloud Firestore, Cloud Functions (Node.js)
AI	Google Gemini API (REST) via gemini_chat_service.dart
Email	Gmail SMTP through Firebase Cloud Functions (nodemailer)
Calendar	Google Calendar API + OAuth 2.0
PDF	Syncfusion Flutter PDF Viewer
Fonts	Google Fonts: Outfit (UI), Figtree (headers)
State	StatefulWidget, ValueNotifier, StreamBuilder, service singletons
Persistence	SharedPreferences (theme), Firestore (user data)

Why Flutter?

Single codebase runs on mobile, web, and desktop. Fast UI development with hot reload. Strong ecosystem for Firebase and Google services integration.

Why Firebase?

Provides authentication, real-time database, serverless functions, and security rules without managing our own backend servers. Scales automatically and integrates natively with Flutter.

5. Application Architecture

Project structure (lib/)

- Pages/ - UI screens (Welcome, Login, Home, Library, BookDetail, etc.)
- services/ - Business logic (auth, library, AI chat, calendar, email)
- models/ - Data classes (Book, Feedback, Preferences, ChatMessage)
- theme/ - Centralized colors, ThemeData, typography, theme switching
- widgets/ - Reusable components (auth scaffold, calendar dialog)

Data flow example: updating reading progress

1. User moves progress slider on BookDetailPage
2. library_service.dart writes to Firestore: users/{uid}/library/{bookId}
3. Firestore real-time listener on LibraryPage receives update
4. UI rebuilds automatically with new progress percentage

Auth flow

Welcome -> Login/Signup -> (optional) Reading Preferences onboarding -> AuthGate checks Firebase Auth session -> HomePage. AuthGate uses StreamBuilder on authStateChanges() to react to login/logout instantly.

6. Firebase Backend

Firestore collections

- books/{bookId} - Global catalog (title, author, cover, PDF URL, pages)
- books/{bookId}/feedbacks - User reviews and star ratings
- users/{uid} - Profile (name, email)
- users/{uid}/library/{bookId} - Personal shelf (status, progress)
- users/{uid}/favorites/{bookId} - Favorited books
- preferences/{uid} - Genres, theme, pace from onboarding
- email_index/{email} - Prevents duplicate sign-ups

Cloud Functions (Node.js)

- sendAuthEmail - Sends verification codes and welcome emails via Gmail
- sendPasswordResetEmail - Sends password reset links

Security

firestore.rules enforces that users can only read/write their own data. Books catalog is readable by authenticated users. Feedback requires auth. Email index is server-managed to prevent account duplication attacks.

7. UI / UX & Theme System

The app uses a unified design system with Light and Dark modes. All screens read colors from a central AppPalette via AppColors.of(context). Users can choose Light, Dark, or System mode in Settings; preference is saved with SharedPreferences.

Design tokens

- Primary brand color: Indigo (#4F46E5 light / #6366F1 dark)
- Typography: Outfit for body text, Figtree for page headers
- Consistent spacing, border radius, card shadows, and elevation
- WCAG-friendly contrast: off-white surfaces, not pure black/white

Screens to show theme switching

Open Settings -> Theme section -> toggle Light / Dark / System. Navigate to Library, Discover, and Profile to show consistent theming across the app.

8. Recommended Live Demo (5-8 minutes)

1. Welcome screen - explain the app's purpose and tagline
2. Sign in (Google or email) - show authentication
3. Home dashboard - Continue Reading, Want to Read, streak
4. Discover tab - browse books by genre
5. Tap a book -> Book Detail - progress, reviews, Start Reading
6. Open PDF reader briefly - in-app reading experience
7. Library tab - Reading / Want to Read / Completed filters
8. Ask AI - ask 'What should I read next?' or 'Rate Atomic Habits'
9. Friends tab - social reading feed (if demo account has data)
10. Settings - Theme picker, Calendar link, Reading goals link

Run the app

```
git clone https://github.com/Aliraya26/BooqlyProject.git
cd BooqlyProject
flutter pub get
flutter run -d chrome --web-hostname=localhost --web-port=54141
```

9. Likely Professor Questions & Answers

Q: What makes your project different from Goodreads or Kindle?

A: Booqly combines library management, in-app PDF reading, AI recommendations grounded in the user's own library, calendar-based reading reminders, and social features in one Flutter app with a custom Firebase backend.

Q: How does the AI assistant work?

A: We call Google Gemini API with a system prompt that includes the user's genres, current library, book catalog, and review data. The AI answers in context. Users can also upload book cover images for visual identification.

Q: How is data stored and secured?

A: Firebase Firestore with security rules. Each user can only access their own library, preferences, and notes. Authentication via Firebase Auth (email + Google).

Q: What was the hardest technical challenge?

A: Integrating multiple Google services (Auth, Calendar, Gemini) with consistent OAuth configuration across web and mobile, plus building a unified theme system that works across 20+ screens.

Q: What is not fully implemented?

A: ISBN barcode scan and manual book entry are partially built. Some dashboard stats use sample UI data. Calendar reminders work best on web/Android.

Q: How did you test the app?

A: Manual testing on Chrome web and Flutter emulators. Firebase console for data verification. Cloud Function logs for email delivery testing.

Q: What would you add next?

A: Push notifications via FCM, offline reading cache, audiobook support, reading clubs, and full ISBN lookup integration.

10. Closing Summary

Booqly demonstrates full-stack mobile development skills: Flutter UI, Firebase backend, third-party API integration (Gemini, Google Calendar), email automation via Cloud Functions, real-time data sync, and a production-quality theme system.

Key numbers to mention

- 60+ Dart source files across pages, services, models, and theme
- 5 main navigation tabs + 15+ feature screens
- Cross-platform: Android, iOS, Web, Windows, macOS, Linux
- Firebase project: booqlyapp-83777
- GitHub: github.com/AliRaya26/BooqlyProject

Suggested closing line

"Booqly shows how modern cross-platform tools and cloud services can turn reading from a passive habit into an intelligent, social, and motivating experience - all from a single Flutter codebase."

Tip: Practice the demo flow twice before presenting. Have a test account logged in with books in your library so Continue Reading and AI chat have real data to show.